



Instituto Infnet

Análise e Segurança de Agentes de IA: Projeto de Bloco

Etapa 3 e 4

Professor: Tiago Cariolano de Souza Xavier

Email : tiago.xavier@prof.infnet.edu.br

OWASP Top 10



OWASP Top 10

- OWASP — Open Worldwide Application Security Project: mantém projetos e materiais sobre segurança de aplicações
- OWASP Top 10: lista de 10 categorias de riscos considerados especialmente relevantes em aplicações web
 - Não deve ser entendido como uma lista de dez ataques específicos
 - **Categorias:**
 - **A01 Broken Access Control**
 - A02 Security Misconfiguration
 - **A03 Software Supply Chain Failures**
 - A04 Cryptographic Failures
 - **A05 Injection**
 - A06 Insecure Design
 - A07 Authentication Failures
 - A08 Software or Data Integrity Failures
 - A09 Security Logging and Alerting Failures
 - A10 Mishandling of Exceptional Conditions



Entrada de campos não previstos

A01 Broken Access Control

Entrada de campos não previstos

- API define quais campos espera receber

- Exemplo:

- `from pydantic import BaseModel`

- ```
class PredictionRequest(BaseModel):
 text: str
```

- API espera somente:

- {

- `"text": "Meu pedido não chegou"`

- }



# Entrada de campos não previstos

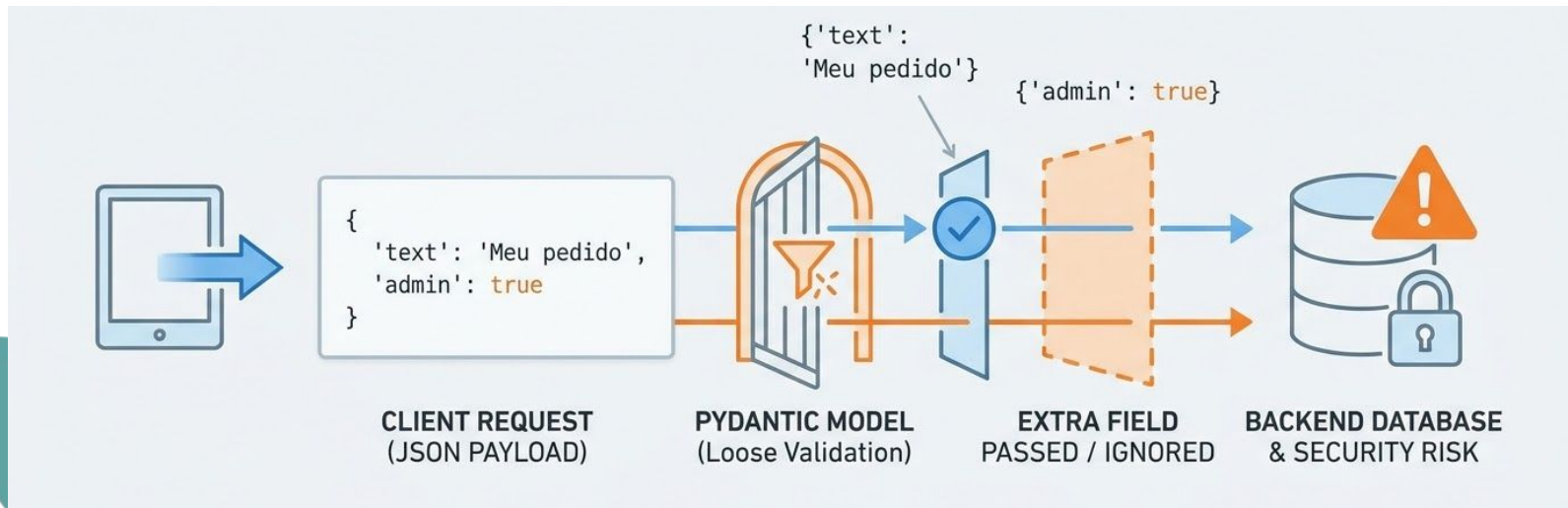
- Cliente controla o conteúdo da requisição e pode tentar enviar campos adicionais:

- ```
{  
    "text": "Meu pedido não chegou",  
    "admin": true,  
    "role": "administrator"  
}
```

- ***Se a aplicação não rejeitar explicitamente campos desconhecidos, dados que não fazem parte do contrato da API podem ser enviados e eventualmente utilizados de maneira indevida***

Entrada de campos não previstos

- Problema:



Entrada de campos não previstos

- Solução: configurar o parâmetro extra:forbid no modelo Pydantic:
 - `from pydantic import BaseModel, ConfigDict`

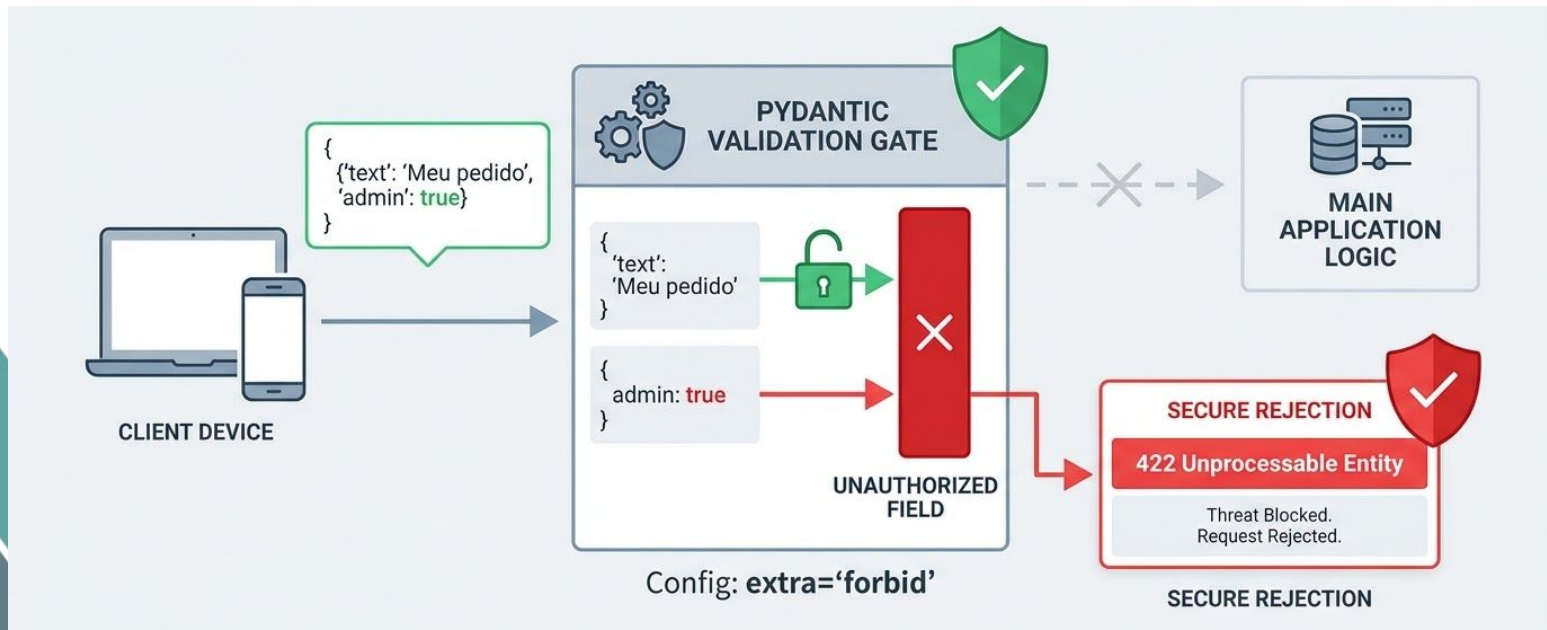
```
class PredictionRequest(BaseModel):  
    model_config = ConfigDict(extra="forbid")
```

```
text: str
```

Two overlapping geometric shapes, a teal parallelogram and a dark grey parallelogram, positioned in the bottom-left corner of the slide.

Entrada de campos não previstos

- Solução:



SQL Injection

A05 Injection

Tutorial SQLite

- `conn = sqlite3.connect(database)`
 - `database`: Caminho do arquivo `.db`. Cria arquivo, caso não exista
 - `conn`: conexão ativa com o banco
- `cursor = conn.cursor()`
 - `cursor`: Objeto que permite realizar consultas SQL
- `cursor.execute(sql)`
 - `sql`: consulta SQL a ser executada no banco de dados
- `results = cursor.fetchall()`
 - Retorna lista de tuplas com as linhas da consulta realizada
- `conn.commit()`
 - Salva as operações realizadas
- `conn.close()`
 - Fecha a conexão

SQLite

- Exemplo: criar tabela user com id, username, password
 - `import sqlite3`
 - `conn = sqlite3.connect('database.db')`
 - `cursor = conn.cursor()`
 - `cursor.execute('''`
 - `CREATE TABLE IF NOT EXISTS user (`
 - `id INTEGER PRIMARY KEY AUTOINCREMENT,`
 - `username TEXT NOT NULL UNIQUE,`
 - `password TEXT NOT NULL)''')`
 - `conn.commit()`
 - `conn.close()`

SQLite

- **Exemplo: inserir dois usuários**

- `cursor.execute("INSERT INTO user (username, password) VALUES ('admin', 'admin')")`
- `cursor.execute("INSERT INTO user (username, password) VALUES ('jose', '1234')")`

- **Exemplo: listar todos os usuários:**

- `result = cursor.execute("SELECT * FROM user")`
- `for line in result:`
 `print(line)`

- **Exemplo: listar usuário pelo id:**

- `cursor.execute("SELECT * FROM user WHERE id = 1")`
`cursor.fetchall()`



SQL Injection

- Montagem manual de SQL pode gerar brechas de segurança

- Exemplo:

- ```
username = input("Username: ")
query = ("SELECT * FROM users " f"WHERE username = '{username}'")
```

- Cenário válido: usuário digitar joao

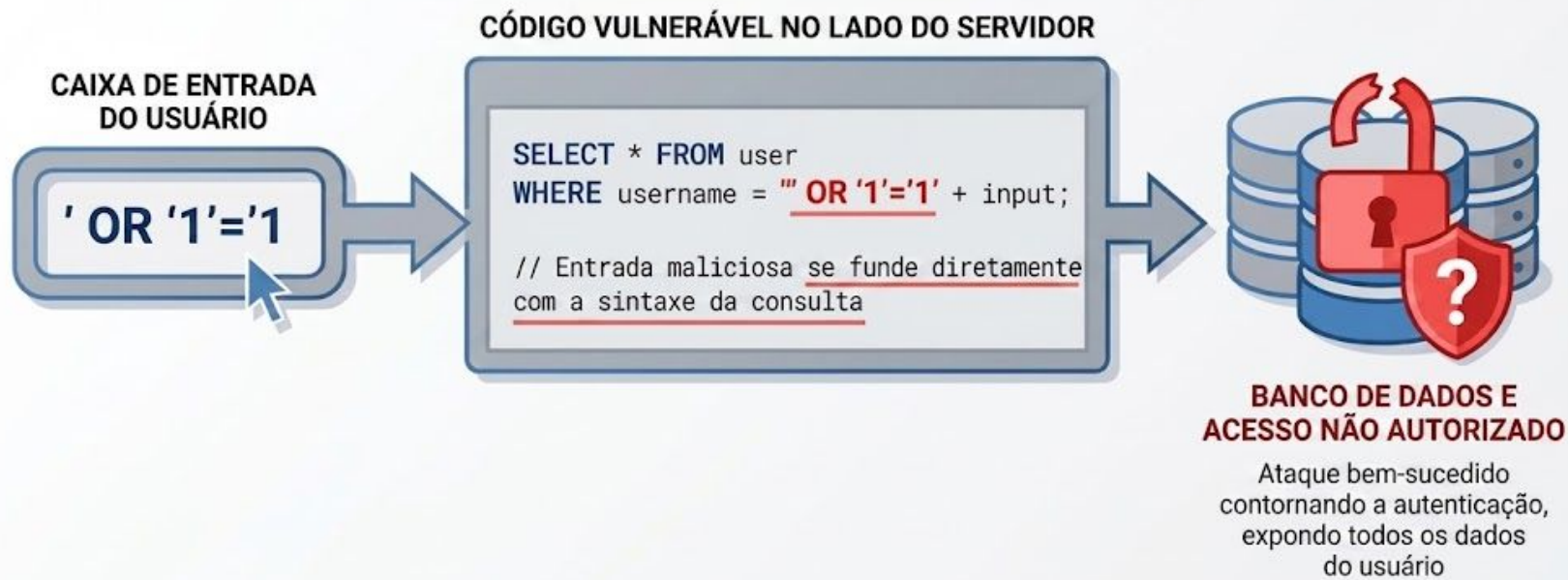
- ```
SELECT * FROM users WHERE username = 'joao'
```

- Cenário malicioso: atacante envia: ' OR '1'='1'

- ```
SELECT * FROM users WHERE username = '' OR '1'='1'
```



# SQL Injection



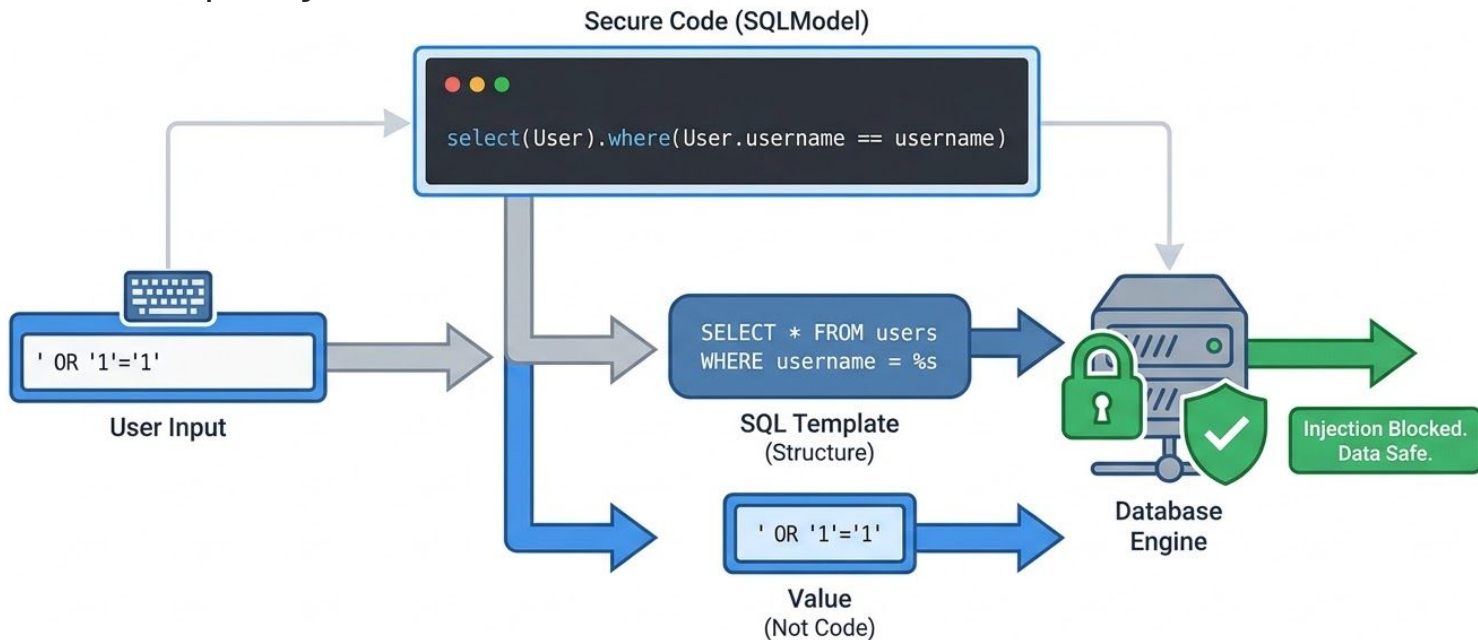
# SQLModel

- **SQLModel:** permite acessar tabelas do banco de dados utilizando classes e objetos Python, sem precisar escrever comandos SQL manualmente
- **Representação SQLModel**
  - Tabela **user** no BD com três colunas - **id, username, password**
  - SQLModel:

```
■ from sqlmodel import SQLModel, Field
 class User(SQLModel, table=True):
 id: int | None = Field(
 default=None,
 primary_key=True
)
 username: str
 password: str
```

# SQLModel

- SQLModel constrói a consulta
- Campos são tratados como valor, e não como código SQL
  - Separação entre COMANDO SQL e DADO



# SQLModel: exemplos

- **Buscar todos os usuários:**

- ```
from sqlmodel import Session, select

with Session(engine) as session:
    statement = select(User).where(User.username == 'joao')
    user = session.exec(statement).first()
```

- **Buscar usuário por ID:**

- ```
statement = select(User).where(User.id == user_id)
user = session.exec(statement).first()
```



# Verificação de Ownership

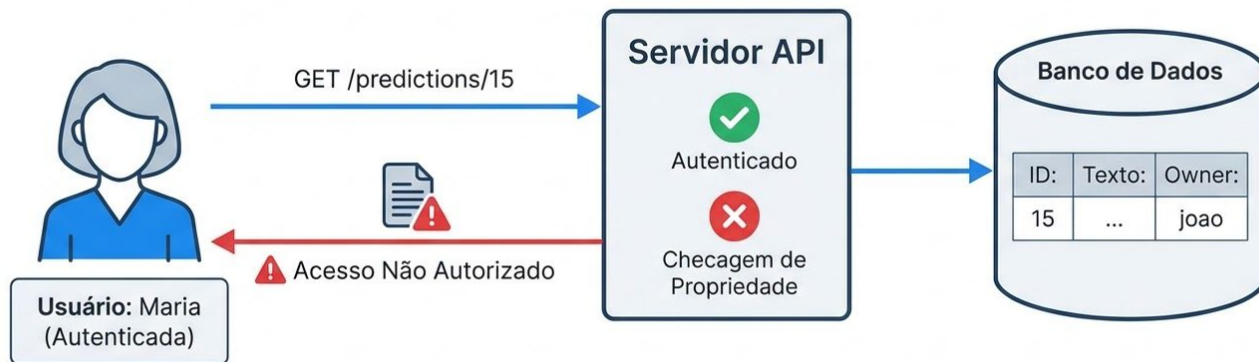
**A01 Broken Access Control**

# Vulnerabilidade BOLA

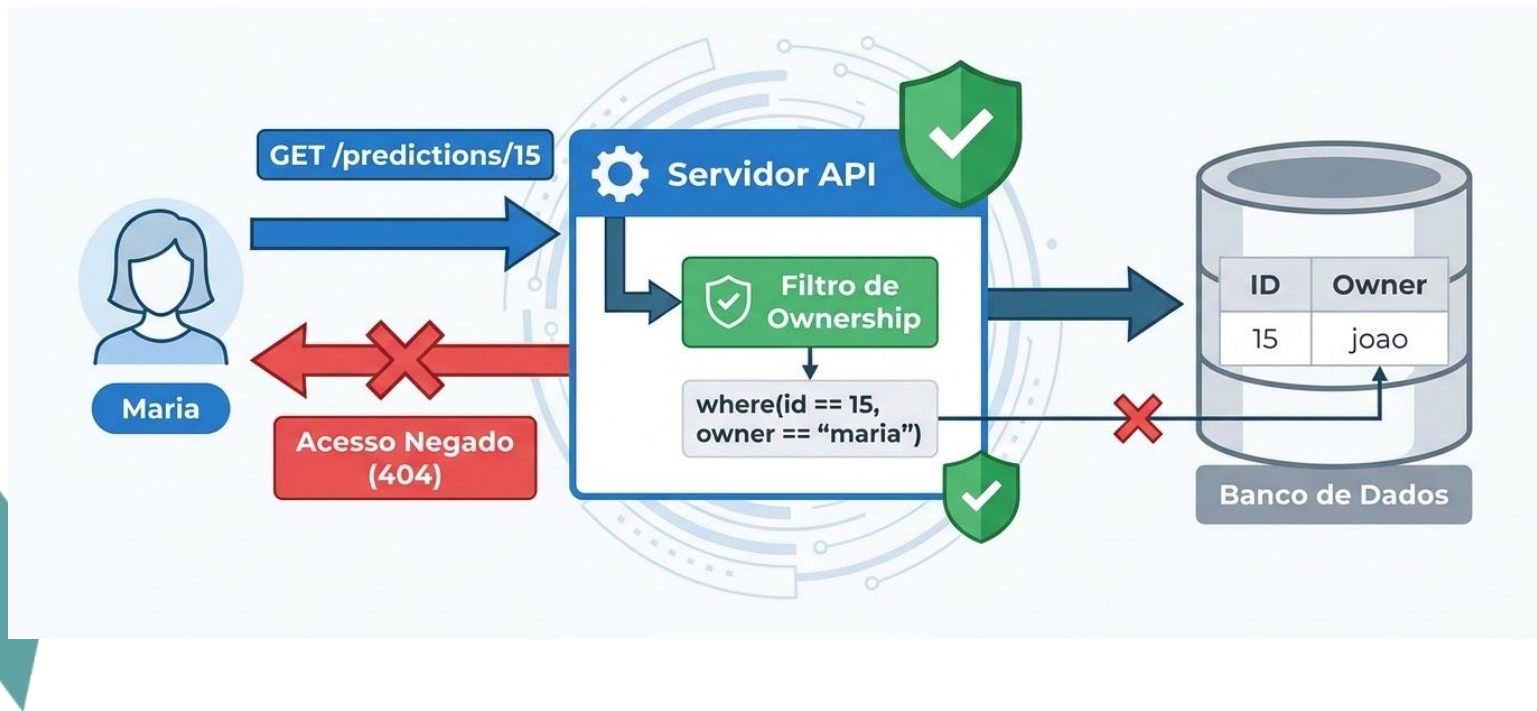
- **BOLA — Broken Object Level Authorization:** vulnerabilidade que ocorre quando a aplicação permite que um usuário acesse um objeto sem verificar se ele possui autorização sobre aquele objeto
- **Exemplo:**
  - **Premissas do sistema:**
    - Prediction 15 → owner = joao
    - Usuário autenticado → maria
  - **Se Maria conseguir acessar tem-se vulnerabilidade BOLA:**
    - GET /predictions/15
- **Autenticação/Autorização:**
  - **Autenticação:** processo utilizado para descobrir quem é o usuário
    - JWT
  - **Autorização:** processo utilizado para determinar o que o usuário pode acessar ou fazer
  - **Ownership:** quem é o proprietário de determinado recurso



# Vulnerabilidade BOLA



# Verificação de ownership



# Verificação de ownership para Predictions

- Tabelas

- prediction: id, text, intent, owner\_id
- user: id, username, password

- Modelos SQLAlchemy:

- ```
class User(SQLModel, table=True):  
    id: int | None = Field(default=None, unique=True, primary_key=True)  
    username: str  
    password: str
```

- ```
class Prediction(SQLModel, table=True):
 id: int | None = Field(default=None, primary_key=True)
 text: str
 intent: str
 owner_id: int = Field(foreign_key="user.id")
```

# Verificação de ownership para Predictions

- Tabelas

- prediction: id, text, intent, owner\_id
- user: id, username, password

- Modelos SQLAlchemy:

- ```
class User(SQLModel, table=True):  
    id: int | None = Field(default=None, unique=True, primary_key=True)  
    username: str  
    password: str
```

- ```
class Prediction(SQLModel, table=True):
 id: int | None = Field(default=None, primary_key=True)
 text: str
 intent: str
 owner_id: int = Field(foreign_key="user.id")
```

# Verificação de ownership para Predictions

- Módulo `dependencies.py`: armazena funções auxiliares
  - Função para obter usuário atual:

```
def get_current_user(username: str = Depends(validar_token),
 session: Session = Depends(get_session)):
 statement = select(User).where(User.username == username)
 user = session.exec(statement).first()
 if user is None:
 raise HTTPException(status_code=401, detail="Usuário não encontrado")
 return user
```

Função `validar_token` é a função implementada para validação jwt que recebe o token, decodifica ele e retorna o username

Função `get_session` é uma função auxiliar que cria uma sessão sqlite para acessar o banco de dados com `SQLModel`.

# Verificação de ownership para Predictions

- Módulo auxiliar `database.py`:

- ```
from sqlalchemy import create_engine, Session
DATABASE_URL = "sqlite:///database.db"
engine = create_engine(DATABASE_URL)
def get_session():
    with Session(engine) as session:
        yield session
```

